Typing function parameters

Today

Welcome to your next lesson: Typing function parameters.

I'm setting up your personalized lesson, building on what you've just learned and what's next for you.

Step by step, you're moving forward.

Ron, we're going to extend what you just learned about types to function parameters in TypeScript — but in tiny steps 👍

Imagine a plain JavaScript function:

Ts
```
function greet(name) {
  console.log("Hello " + name);
}
```
In TypeScript, we can tell the function that name must be a string, like this:

Ts
```
function greet(name: string) {
  console.log("Hello " + name);
}
```
That : string is a parameter type annotation — it's a little label that says "the argument for name must be a string."

Quick check, just your intuition for now:

If someone calls this function like greet(42), what do you expect TypeScript to do during development?

A) Accept it silently
B) Complain that the argument type is wrong

You don't need to explain yet — just answer A or B.

B

Nice 🎯 — B is right. TypeScript will complain because 42 is not a string. That's the power of typed parameters: they catch bad calls early.

Let's do a first tiny hands-on step.

Open the TypeScript code editor for this lesson.
In a fresh file, type this function (no types yet):
Ts
```
function greet(name) {
  console.log("Hello " + name);
}

greet("Ron");
```
Hit Run to make sure it works.
Then change the function to add a type to the parameter:
Ts
```
function greet(name: string) {
  console.log("Hello " + name);
}

greet("Ron");
```

Run it again, then hit Submit code.

Do that now, and I'll look at your submission before we go to the next step.

Typescript
```
function greet(name: string) {
  console.log("Hello " + name);
}

greet("Ron");
```
Nice job, Ron — that's a perfectly typed parameter and a clean run ☑

You just used a parameter type annotation: name: string.

That tells TypeScript: "whoever calls greet must pass a string for name."

Next, I want you to see TypeScript complain.

In the same editor, under your existing code, add this new call:

Ts
```
greet(123);
```
So the full code ends with both:

Ts
```
greet("Ron");
greet(123);
```
Then:

Hit Run (or just hover / look at the editor squiggles) and notice what TypeScript says.
Then hit Submit code so I can see the version with the wrong argument.

Do that now, and then we'll talk about what the error is really telling you.

Typescript
```
function greet(name: string) {
  console.log("Hello " + name);
}

greet("Ron");
greet(123);
```
Perfect 👍

TypeScript's message:

Argument of type 'number' is not assignable to parameter of type 'string'.

matches exactly your annotation name: string.

So:

Parameter in definition: name: string
Argument in bad call: 123 (a number)
TypeScript: "Nope, that doesn't match."

This is argument type checking at development time.

Quick understanding check (just answer in words):

In your own words, what is TypeScript enforcing when it complains about greet(123)?

You can keep it simple, like: “It’s making sure that …”

its enforcing that the the type of the argument passed to greet is of type string as declared

Exactly 👌

TypeScript is making sure that every call to greet passes a string, because you declared name: string. So greet("Ron") is fine, greet(123) is not.

Next step: multiple parameters.

Let’s make a simple math function with two typed parameters.

In the editor, below your current code, add this new function and a call:

Ts
```
function add(a: number, b: number) {
  console.log(a + b);
}

add(5, 7);
```
Then:

Run it to see the output.
Hit Submit code with this added.

Do that, and we’ll use it to explore wrong/missing/extra arguments.

Typescript
```
function greet(name: string) {
  console.log("Hello " + name);
}

greet("Ron");

function add(a: number, b: number) {
  console.log(a + b);
}

add(5, 7);
```
Great, Ron — you now have:

Ts
```
function add(a: number, b: number) {
  console.log(a + b);
}

add(5, 7);
```
Here, the contract is: “both a and b must be numbers.”

Let’s test how TypeScript reacts to a wrong type on one of these.

In the editor, change just the call to:

Ts
```
add(5, "7");
```
Leave the function definition the same. Then:

Run it (or just look at the error).
Submit the code with that change.

After you do that, I'll ask you to explain what TypeScript is complaining about.

Typescript
```
function greet(name: string) {
  console.log("Hello " + name);
}

greet("Ron");

function add(a: number, b: number) {
  console.log(a + b);
}

add(5, "7");
```
Nice, that's exactly the kind of error we want to see 👀

TypeScript says:

Argument of type 'string' is not assignable to parameter of type 'number'.

Let's decode that in very clear terms:

Function contract: a: number, b: number
Call: add(5, "7")
Problem: "7" is a string, but b must be a number

So TypeScript is catching a type mismatch before the code runs.

Quick confirmation:

Does it make sense that this error is happening because the second argument "7" doesn't match b: number?

You can just say "yes" or "not really".

Yes, that makes sense

Good 👍

So far you've seen TypeScript catch:

A wrong type (greet(123), add(5, "7"))

Now I want to zoom in on argument count — missing vs extra arguments.

Take your current add:

Ts
```
function add(a: number, b: number) {
  console.log(a + b);
}
```
In the editor, change the call to pass only one argument:

Ts
```
add(5);
```
Then:

Run it / check the error.
Submit the code with that change.

After that, I'll ask you what the error is really saying in your own words.

Typescript
```
function greet(name: string) {
  console.log("Hello " + name);
}

greet("Ron");

function add(a: number, b: number) {
  console.log(a + b);
}

add(3);
```
Nice, you just triggered the argument-count check ☑

The error says:

Expected 2 arguments, but got 1.

Visually, your function wants:

Ts
```
add(a: number, b: number)
//        ^        ^
//       1st      2nd parameter
```
But your call only gives:

Ts
```
add(3);
//   ^
//  just 1 argument
```
So TypeScript is enforcing: "You promised I always get two numbers. You only gave me one."

Quick check for you:

If you change the call to this:

Ts
```
add(3, 4, 5);
```
what do you expect TypeScript to complain about?

A) Wrong types
B) Too many arguments

Just pick A or B.

b

Correct again 🎯 — B) Too many arguments.

Because the contract is still:

Ts
```
function add(a: number, b: number) { ... }
//           ^          ^
//        2 parameters
```
So a call like add(3, 4, 5) breaks the agreement on how many arguments, even if each is a number.

I want to check your understanding with a tiny scenario, then we'll talk about how this helps usability.

Imagine this function:

Ts
```
function labelUser(name: string, age: number) {
  console.log(name + " is " + age + " years old");
}
```
Would TypeScript be happy or unhappy with each call? Think it through slowly:

```
labelUser("Ron", 30)
labelUser(30, "Ron")
labelUser("Ron")
```

Pick which ones are valid calls: 1, 2, and/or 3?

Just answer with something like "only 1" or "1 and 3", etc.

1 is valid. 2 is not, its sending a number instead of a string and a string instead of a number. 3 is invalid as its only sending 1 argument

Exactly right on all three 💪

labelUser("Ron", 30) ☑ correct types, correct count
labelUser(30, "Ron") ✗ types swapped
labelUser("Ron") ✗ missing the age argument

That shows you understand both types and argument counts as part of the function's contract.

Last piece we need for this lesson is you explaining the benefit in your own words.

Think of a teammate who only knows plain JavaScript.

In 1–3 simple sentences:

Why do typed parameters (like name: string, age: number) make your code easier to use and less buggy?

using typed parameters enforces that the code is correctly designed to produce the desired result. Its enforced befoe the code runs so that the error doesnt happen in production, potentialy causing incorrect results that the end user is relying on to be accurate

That's a strong explanation, Ron 👌

You hit the key ideas:

Typed parameters act like a design contract: they describe exactly what inputs the function is built for.
TypeScript checks that contract before the code runs, so:
you catch wrong types and wrong argument counts early,
you avoid subtle wrong-results bugs in production that users might trust.

That covers all three goals for this lesson:

Add parameter types ☑
Interpret wrong/missing/extra arguments ☑
Explain how typed parameters improve usability and correctness ☑

You've mastered typed function parameters for this level 🎯

Ready to wrap up this lesson and move on to the next one, or do you want one more quick practice challenge first?

Move on to the next lesson

Nice work, Ron 🎉

Quick recap of what you can now do:

You can add parameter type annotations like name: string or age: number to make a clear contract for each function.
You can read and interpret TypeScript errors about:
wrong types (Argument of type 'string' is not assignable to parameter of type 'number')
missing arguments (Expected 2 arguments, but got 1)
extra arguments (Expected 2 arguments, but got 3)
You understand that typed parameters catch mistakes before runtime, which protects users from bad or misleading results.

You're ready for the next step in TypeScript 👏

When you're set, hit Next lesson in the Learn tab, and I'll meet you there.

```
function greet(name: string) {
  console.log("Hello " + name);
}

greet("Ron");

function add(a: number, b: number) {
  console.log(a + b);
}

add(3);
```